

Cours de Programmation en C – EI-2I

Travaux Dirigés

TD8

Exercice 1 – Allocation de tableau

On considère le programme suivant:

```
#include <stdio.h>
#include <stdlib.h>

#define TAILLE 1024

/* affiche info sur le tableau */
void aff_info (int *p)
{
    printf("%p->p[0]=%i\n", p, *p);
}

/* créé nouveau tableau */
int * tab_nouv (int n)
{
    int t[TAILLE];
    int i;
    for (i=0; i<TAILLE; ++i)
        t[i]=n;
    return t;
}

int main ()
{
    int t1[TAILLE] ;
    int t2[1] ;
    int t3[TAILLE] ;
    int t4[1] ;
    int *t5 = tab_nouv(4);
    int *t6 = tab_nouv(2);

    /* Affiche les adresses des tableaux statiques et de deux "tableaux" créés par tab_nouv()
       */
    printf("Tableaux_statiques:\n");
    aff_info(t1);
    aff_info(t2);
    aff_info(t3);
    aff_info(t4);
    printf("Tableaux_dynamiques_(?):\n") ;
    aff_info(t5);
    aff_info(t6);
    return EXIT_SUCCESS;
}
```

- Quel va être son comportement et sa sortie? Que se passe-t-il pour les tableaux `t5` et `t6` ?
- Comment modifier le programme (et la fonction `tab_nouv`) pour obtenir un comportement correct ?

Solution:

- On obtient le résultat suivant:

```
Tableaux statiques:
0x7fff5fbfe910 -> p[0]=0
0x7fff5fbff930 -> p[0]=0
0x7fff5fbfd910 -> p[0]=0
0x7fff5fbff920 -> p[0]=0
Tableaux dynamiques (?):
0x7fff5fbfc8f0 -> p[0]=2
0x7fff5fbfc8f0 -> p[0]=2
```

Les adresses des tableaux t1, t2, t3 et t4 sont différentes. Par contre, on obtient les mêmes adresses pour p5 et p6.

En effet, le tableau statique créé dans la fonction `tab_nouv` est "détruit" à la sortie de la fonction. Donc la création de p6 se fait sur p5.

- Il faut donc faire une allocation dynamique dans `tab_nouv` (avec un `malloc`) et détruire à la fin du `main`.

```
#include <stdio.h>
#include <stdlib.h>

#define TAILLE 1024

/* affiche info sur le tableau */
void aff_info (int *p)
{
    printf("%p->p[0]=%i\n", p, *p);
}

/* crée nouveau tableau */
int * tab_nouv (int n)
{
    int i;
    int *t = (int*) malloc( TAILLE*sizeof(int) );
    for (i=0; i<TAILLE; ++i)
        t[i]=n;
    return t;
}

int main ()
{
    int t1[TAILLE] ;
    int t2[1] ;
    int t3[TAILLE] ;
    int t4[1] ;
    int *t5 = tab_nouv(4);
    int *t6 = tab_nouv(2);

    /* Affiche les adresses des tableaux statiques et de deux "tableaux" créés par tab_nouv() */
    printf("Tableaux statiques:\n");
    aff_info(t1);
    aff_info(t2);
    aff_info(t3);
    aff_info(t4);
    printf("Tableaux dynamiques(?):\n") ;
    aff_info(t5);
    aff_info(t6);

    /* on désalloue */
    free(t5);
    free(t6);
    return EXIT_SUCCESS;
}
```

On peut même constater que l'allocation ne se fait pas au même endroit (pile/tas) :

```
0x7fff5fbfe880 -> p[0]=0
0x7fff5fbff8a0 -> p[0]=1606416568
0x7fff5fbfd880 -> p[0]=0
0x7fff5fbff890 -> p[0]=0
Tableaux dynamiques (?):
0x100801000 -> p[0]=4
0x100801000 -> p[0]=2
```

Exercice 2 – Allocation de mémoire : tableau et chaines

On cherche à stocker un certain nombre de fiches d'un carnet d'adresses. Les données sont sauvegardées dans un fichier texte qui contient:

- le nombre de fiches

- chaque fiche, c'est-à-dire ;
 - le nom

 - le prénom

 - un age

(pour simplifier, on suppose qu'un simple espace sépare tous ces champs)

Question 1

Créer un type structuré `t_fiche` correspondant à une fiche.

Question 2

Écrire une procédure `remplirFiche` qui accepte un `FILE*` et un `t_fiche*` et qui remplit la fiche en question avec les valeurs lues dans le fichier (que l'on suppose ouvert à l'endroit de lecture d'une fiche). Pensez à bien allouer ce qui doit l'être.

Question 3

Écrire une procédure `remplirTableauFiches` qui accepte un nom de fichier et qui alloue et remplit un tableau de fiches à partir de la lecture du fichier (il faudra récupérer le tableau et la taille du tableau). On se servira de la procédure précédente.

Question 4

Écrire une procédure qui désalloue la mémoire allouée pour le tableau de fiches (et pour le contenu de chacune des fiches).

Solution:

```
#ifndef __FICHE_H__
#define __FICHE_H__

#include <stdio.h>

/* déclaration du type "fiche" */
typedef struct
{
    char* nom;
    char* prenom;
    int age;
} t_fiche;

/* création d'un tableau de fiches
il faudra désallouer ce tableau après */
t_fiche* creerTableauFiches( char* nomFichier, int* pNbFiches);

/* remplit une fiche depuis un fichier */
void remplirFiche( FILE* fichier, t_fiche* pFiche);

/* désalloue les chaines de caractères d'une fiche */
void detruireFiche( t_fiche* pFiche);

/* détruit le tableau de fiches */
void detruireTableauFiches( t_fiche* fiches, int nbFiches);

/* affiche une fiche */
void afficherFiche( t_fiche fiche);

#endif
```

Solution:

```
#include "fiche.h"
#include <stdlib.h>
#include <string.h>

/* remplit une fiche depuis un fichier */
void remplirFiche( FILE* fichier, t_fiche* pFiche)
{
    char nom[50];
    char prenom[50];
    int age;

    /* lecture des valeurs */
    fscanf( fichier, "%s%s%d", prenom, nom, &age);

    /* allocation et recopie */
    pFiche->nom = (char*) malloc( strlen(nom)+1 );
    if (pFiche->nom)
        strcpy( pFiche->nom, nom);
    pFiche->prenom = (char*) malloc( strlen(prenom)+1 );
    if (pFiche->prenom)
        strcpy( pFiche->prenom, prenom);
    pFiche->age = age;
}

/* création d'un tableau de fiches
il faudra désallouer ce tableau après */
t_fiche* creerTableauFiches( char* nomFichier, int* pNbFiches)
{
    int i;
    t_fiche* fiches = NULL;
    t_fiche* pFicheCour;
    FILE* fichier = fopen( nomFichier, "r");

    if (fichier)
    {
        /* lecture du nbre de fiches */
        fscanf( fichier, "%d", pNbFiches);

        /* allocation du tableau de t_fiches */
        fiches = (t_fiche*) malloc ( *pNbFiches * sizeof(t_fiche) );
        pFicheCour = fiches;

        /* lecture une à une des fiches */
        if (fiches)
        {
            for( i=0; i<*pNbFiches; i++)
            {
                remplirFiche( fichier, pFicheCour++);
            }
        }
        /* fermeture du fichier */
        fclose( fichier);
    }

    return fiches;
}

/* désalloue les chaines de caractères d'une fiche */
void detruireFiche( t_fiche* pFiche)
{
    if (pFiche->nom)
        free( pFiche->nom);
    if (pFiche->prenom)
        free( pFiche->prenom);
    pFiche->nom = NULL;
    pFiche->prenom = NULL;
}

/* détruit le tableau de fiches */
void detruireTableauFiches( t_fiche* fiches, int nbFiches)
{
    int i;
    t_fiche* pFicheCour = fiches;
    if (fiches)
    {
        /* désalloue le contenu de chaque partie */
        for( i=0; i<nbFiches; i++)
        {
            detruireFiche( pFicheCour++ );
        }
        /* désalloue le tableau de fiches */
        free( fiches );
    }
}

/* affiche une fiche */
void afficherFiche( t_fiche fiche)
{
    printf( "%s%s_a_%d_ans\n", fiche.nom, fiche.prenom, fiche.age);
}

int main()
{
    t_fiche* fiches;
    int i, nbFiches;

    /* création du tableau de fiches */
    fiches = creerTableauFiches( "fiches.txt", &nbFiches);

    /* affiche les fiches */
    for( i=0; i<nbFiches; i++)
        afficherFiche( fiches[i]);

    /* destruction des fiches */
    detruireTableauFiches( fiches, nbFiches);

    return EXIT_SUCCESS;
}
```